

[DHS/2022/Q1]

Task 1

Singapore's National Day celebrations mark the founding of our nation. Our National Day Parade (NDP) comprises multiple marching contingents representing the facets of our Nation and our Defence.

NDP marching contingents usually consist of individuals between 1.5 m to 1.8 m tall.

Task 1.1

Write a function `generate_heights(size)` which takes in a positive integer `size` and returns a list of `size` random integers between 150 and 180 inclusive. Test the function by calling it with `size 20` as its parameter. Display the returned content.

[5]

Task 1.2

Write a function `mergesort(input_list)` which takes in any given list as `input_list`, uses merge sort to sort in ascending order, and returns the sorted list.

Write a nested function call which uses `generate_heights(size)` from Task 1.1 to generate the list and `mergesort(input_list)` to sort it. Display the returned content from the nested function call.

[6]

[DHS/2022/Q1 Cont']

Parade commanders need an automated way to arrange participants by height for NDP and to inform them where to stand.

Your program will take in participant height (cm) and NRIC data and use a 2D array to generate a contingent formation similar to:

```
Row 0: [[179, 'T0663758C'], [174, 'T0849104E'], [157, 'S6254881Y'], [159, 'S7245932B'], [174, 'S7696370P'], [180, 'T2333665L']]
Row 1: [[178, 'T0223232U'], [172, 'T0230412W'], [153, 'S5334545B'], [155, 'S8547302F'], [174, 'T0434342F'], [178, 'T0343233B']]
Row 2: [[178, 'T0340912U'], [171, 'T0482309S'], [151, 'S5324433F'], [152, 'S2352434Y'], [172, 'T0320135V'], [178, 'T0281099B']]
Row 3: [[177, 'T0433322M'], [167, 'T0867787F'], [150, 'S7166399M'], [150, 'S7572379E'], [168, 'T0254885I'], [178, 'T0442131M']]
Row 4: [[175, 'S6528694I'], [161, 'T0574412P'], [150, 'S6043657K'], [150, 'S6395844J'], [164, 'S7457420P'], [175, 'T0198720F']]
Column: 0      1      2      3      4      5
```

- Note the following formation requirements:
- The shortest participants are centred and positioned nearest to the audience (Row 4), while the tallest are further back and towards the ends (columns 0 and 3)
 - Each row always has an even number of participants
 - There are always 5 rows; number of columns change depending on the number of participants
 - The total number of soldiers for each contingent is always a multiple of 10

Task 1.3

Write a function `arrange(height_list)` which can take in a sorted list of **any** number of height-NRIC pairs and return the above 2D array format. Test `arrange(height_list)` by taking in the list `sorted_heights_nrics` (provided in task1.ipynb) and return the following 2D array

```
[[[178, 'T0281099B'], [168, 'T0254885I'], [171, 'T0482309S'], [179, 'T0663758C']],
 [[178, 'T0442131M'], [164, 'S7457420P'], [167, 'T0867787F'], [178, 'T0340912U']],
 [[175, 'S6528694I'], [159, 'S7245932B'], [161, 'T0574412P'], [175, 'T0198720F']],
 [[174, 'T0849104E'], [155, 'S8547302F'], [157, 'S6254881Y'], [174, 'S7696370P']],
 [[172, 'T0320135V'], [150, 'S7166399M'], [150, 'S7572379E'], [172, 'T0230412W']]]
```

such that each part of the list corresponds to a contingent row shown above.

The table below illustrates how `sorted_heights_nrics` is mapped onto the 2D array. The rows and columns of the table correspond to that of the contingent formation shown at the start of this task. To illustrate where each height-NRIC pair should be placed, the `sorted_heights_nrics` list index of each pair starting from 0 is placed into its corresponding row and column.

		Column			
		0	1	2	3
Row	0	[18]	[8]	[9]	[19]
	1	[16]	[6]	[7]	[17]
	2	[14]	[4]	[5]	[15]
	3	[12]	[2]	[3]	[13]
	4	[10]	[0]	[1]	[11]

Test the function by calling it and displaying the 2D array it returns.
[6]

[DHS/2022/Q1 Cont']

For Task 1.4 and 1.5, you will be extending the program to allow contingent participants to query position coordinates by entering their NRIC and height.

Task 1.4

To speed up data retrieval and to respond to new data privacy requirements (only hash value of NRICs allowed to be stored, not the original NRIC), you will be creating a hash table and transferring data from a contingent 2D array into it.

For this task, you are provided with a modified array as `mod_array` in `task1.ipynb`. The array has been modified to include the position coordinates of each participant:

```
[[[178, 'T0281099B', [0, 0]], [168, 'T0254885I', [0, 1]], [171, 'T0482309S', [0, 2]], [179, 'T0663758C', [0, 3]],  
[[178, 'T0442131M', [1, 0]], [164, 'S7457420P', [1, 1]], [167, 'T0867787F', [1, 2]], [178, 'T0340912U', [1, 3]],  
[[175, 'S6528694I', [2, 0]], [159, 'S7245932B', [2, 1]], [161, 'T0574412P', [2, 2]], [175, 'T0198720F', [2, 3]],  
[[174, 'T0849104E', [3, 0]], [155, 'S8547302F', [3, 1]], [157, 'S6254881Y', [3, 2]], [174, 'S7696370P', [3, 3]],  
[[172, 'T0320135V', [4, 0]], [150, 'S7166399M', [4, 1]], [150, 'S7572379E', [4, 2]], [172, 'T0230412W', [4, 3]]]
```

Initialise `hash_table` as a list representing an empty hash table of `tablesize` 29 with integer `-1` to indicate empty slots. Its operations are functions `hash_funct(key)`, `insert(key, data)` and `search(key, data)`.

- The hash function `hash_funct(key)` uses modulo division by its `tablesize` to obtain the hash value from each NRIC (digits only; exclude first and last alphabet).
- The insert and search operations should also use **linear probing** for collision resolution
- The search function should return strings informing the results of search, such as
 - Found (with collision resolution): Row 4 Col 1
 - Found (no collision resolution): Row 4 Col 2
 - Not found (with collision resolution)
 - Not found (no collision resolution)

Access and transfer the data from the `mod_array` into `hash_table`. Finally, display `hash_table` to be:

```
[-1, -1, [178, [0, 0]], [174, [3, 3]], [168, [0, 1]], [172, [4, 0]], [179,  
[0, 3]], [150, [4, 1]], [172, [4, 3]], [161, [2, 2]], [171, [0, 2]], [175,  
[2, 0]], [164, [1, 1]], [175, [2, 3]], [174, [3, 0]], [150, [4, 2]], [155,  
[3, 1]], [178, [1, 3]], [157, [3, 2]], -1, [167, [1, 2]], [159, [2, 1]], -1,  
-1, -1, -1, [178, [1, 0]], -1, -1]
```

[15]

Task 1.5

Write code to allow a contingent participant to query the program to retrieve their standing position coordinates by entering their NRIC and height. Your output should match the following for an input of S7166399M and 150:

```
Input your NRIC: S7166399M  
Input your height(cm): 150  
Found (with collision resolution): Row 4 Col 1
```

Run and test your code with the above example.

[1]

[DHS/2022/Q1 Cont']

Task 1.6

Write test cases for `search(key, data)` and run the test cases using appropriate data.

To do this, test the function with input values by calling it using the following statement format:

```
print(search(input_value1, input_value2) == expected)
```

The statements should all print `True`.

[4]